



SecureRx

Sistema de Recetas Electrónicas Cifradas

Documentación criptográfica

Guía de defensa para la materia de Criptografía

Algoritmos: ECDSA P-256 + SHA3-256 · AES-128-GCM · RSA-2048-OAEP-SHA256

Hash de password: Argon2id

Sesión: JWT HS256 (60 min)

Esquema: Cifrado híbrido con DEK efímera y wraps por destinatario

Stack backend: FastAPI · SQLAlchemy 2 · MySQL 8 · Python 3.12

Cripto-library: `cryptography` (pyca)

Contenido

1. Resumen ejecutivo
2. Algoritmos elegidos y justificación
3. Esquema híbrido
4. Flujo criptográfico por caso de uso
5. Controles criptográficos críticos
6. Propiedades garantizadas
7. Material privado del sistema
8. Cómo correr las pruebas
9. Cómo ver el flujo en vivo
10. Preguntas probables y respuestas en código
11. Cheatsheet de comandos

1. Resumen ejecutivo

SecureRx es un sistema de recetas electrónicas con tres roles (médico, paciente, farmacéutico) más administrador. Cada receta se **firma digitalmente** con la clave privada del médico, se **cifra una sola vez** con AES-128-GCM bajo una clave simétrica efímera (DEK), y la DEK se **envuelve N veces** con RSA-OAEP — una vez para el paciente y una por cada farmacia autorizada — usando la clave pública RSA de cada destinatario.

Las claves privadas de los usuarios **nunca** tocan el servidor: se entregan al usuario una sola vez en el registro y deben subirse en cada operación que requiera firmar o descifrar. La pertenencia priv ↔ pub registrada se valida en cada request, funcionando como segundo factor implícito.

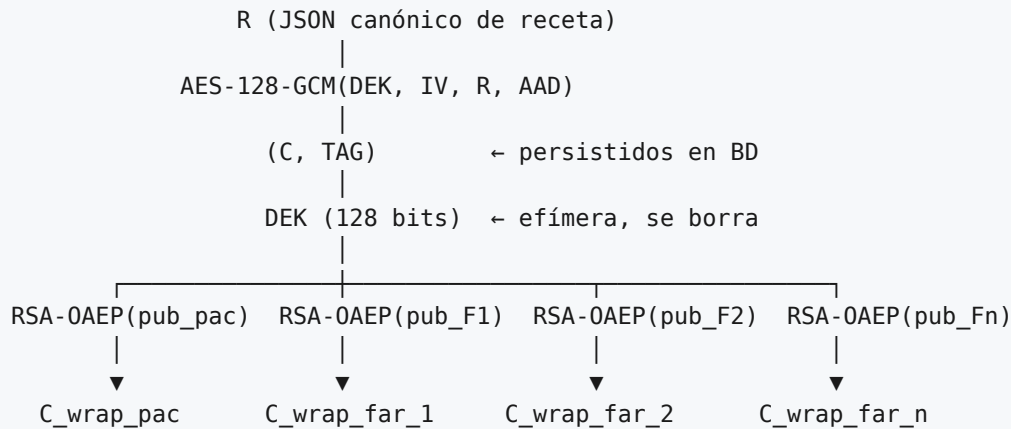
Propiedad fuerte del diseño: aunque un atacante consiga acceso completo al servidor (incluida la base de datos, la CA interna y el secreto del JWT), no puede leer ni una sola receta — la DEK está envuelta con la clave pública RSA del paciente, y la clave privada del paciente no existe en ese disco.

2. Algoritmos elegidos y justificación

Categoría	Algoritmo	Archivo · referencia	Justificación
Firma digital	ECDSA · curva P-256 (SecP256r1) · SHA3-256	<code>services/crypto/signatures.py</code>	Llaves de 256 bits ≈ seguridad de RSA-3072. SHA3 evita la extensión-de-longitud y los problemas teóricos de SHA-2.
Cifrado simétrico	AES-128-GCM (clave 128 bits, IV 96 bits, TAG 128 bits)	<code>services/crypto/aes_gcm.py</code>	AEAD: cifra y autentica en una sola pasada. Aceleración hardware nativa.
Envoltura de clave	RSA-2048-OAEP (SHA-256, MGF1-SHA-256)	<code>services/crypto/rsa_oaep.py</code>	OAEP es CCA-seguro. PKCS#1 v1.5 padece de ataques de Bleichenbacher.
Hash de password	Argon2id	<code>services/crypto/argon2_pw.py</code>	Memory-hard → resistente a GPU/ASIC. Ganador del Password Hashing Competition.
Hash de mensaje	SHA3-256 (huella <code>hash_sha3_hex</code>)	<code>services/canonical_receta.py:140</code>	Familia distinta a SHA-2; sin extensión de longitud.
Sesión	JWT HS256 (60 min)	<code>services/crypto/jwt_service.py</code>	Stateless, secret ≥ 32 chars validado en arranque.
Canonicalización	JSON con <code>sort_keys</code> + separadores compactos	<code>services/crypto/canonical.py</code>	Bytes deterministas → firmas reproducibles entre emisor y verificador.

3. Esquema híbrido

El cuerpo R de la receta (JSON canónico) se cifra **una vez** con AES-128-GCM. La DEK se envuelve con RSA-OAEP **N veces** — una por destinatario autorizado.



La implementación atómica está en `services/receta_cifrado.py:31-50` (función `cifrar_y_envolver`).

3.1 ¿Por qué híbrido y no RSA puro?

- RSA-OAEP-2048 cifra como mucho ≈ 190 bytes. Una receta no cabe.
- RSA es $\sim 1000\times$ más lento que AES por byte.
- Sumar destinatarios solo requiere wraps adicionales (RSA sobre 16 B), no recifrar la receta.

3.2 ¿Qué es el AAD y por qué importa?

AAD (*Additional Authenticated Data*) son metadatos que viajan **en claro** pero que el TAG de GCM **autentica** junto con el ciphertext. En SecureRx el AAD canónico contiene `id_receta`, `id_doctor`, `id_paciente`, `fecha_creacion`, `dispensaciones_permitidas`, `version`.

Resultado: si un atacante con acceso a BD intentara cambiar `medico_id` en la fila SQL, el AAD recomputado en el descifrado diferiría del firmado y el TAG fallaría — la cripto detecta manipulaciones en columnas externas, no solo en el ciphertext.

4. Flujo criptográfico por caso de uso

§4 EMISIÓN — el médico crea una receta

router routers/recetas_crear.py

Paso	Operación	Línea
0	Validar pertenencia de la priv EC del médico	:38 bundle.exigir_ec
§4.1	Construir R canónico (bytes deterministas)	:84-92
§4.2 ★	S_D = ECDSA_Sign(priv_ec_doctor, R)	:104
§4.4	Construir AAD canónico	:111-120
§4.3- §4.7	AES-128-GCM(R, DEK, IV, AAD) + N×RSA-OAEP(DEK)	:126-131 → receta_cifrado.py:42-50
§4.8	Persistir (C, TAG, IV, AAD, S_D, hash, C_wrap_pac)	:140-150
§4.9	1 fila RecetaAccesoFarmacia por farmacia con su C_wrap_far	:152-156

§5 CONSULTA — paciente o farmacia descifran

router routers/recetas_consulta.py

- **Paciente:** GET /recetas/paciente/{id} . Sube su priv RSA en header X-Priv-Keys . Por cada receta: RSA-OAEP-Decrypt → AES-GCM-Decrypt . Líneas :42-67 .
- **Farmacéutico:** idéntico, usando su c_wrap_far . Líneas :120-150 .
- **Médico:** lista sus recetas pero ve "(cifrado)" en el cuerpo — por diseño no se le emite C_wrap_doc . Líneas :74-105 .
- **Verificación pública** (/id/verificar-firmas) : sin priv keys, valida con ecdsa_verify el sello del farmacéutico (cuyo manifiesto sí está en BD).

§6 DISPENSACIÓN — la farmacia entrega el medicamento

router routers/recetas_dispensar.py

Paso	Operación	Línea
0	<code>bundle.exigir_ec</code> + <code>bundle.exigir_rsa</code> del farmacéutico	:140-141
§6.3	DEK = RSA-OAEP-Decrypt(priv_rsa, c_wrap_far)	:154-157
§6.4 ★	R = AES-GCM-Decrypt(DEK, IV, C, TAG, AAD) — TAG falla → 400	:168-173
§6.5 ★	ECDSA_Verify(pub_ec_doctor, R, S_D) ← control central de autenticidad	:185-191
§6.7 ★	S_F = ECDSA_Sign(priv_ec_farm, Sello) — sello digital	:202-211
§6.7+	INSERT <code>EventoDispensacion</code> con manifiesto y firma	:218-222
§6.b	Acuse del paciente: ECDSA-Sign + Verify sobre el mismo manifiesto	:295-303

§8 CANCELACIÓN — el médico cancela una receta

router `routers/recetas_cancelar.py`

Paso	Operación	Línea
0	<code>bundle.exigir_ec</code> del médico	:46
§8.2	Construir <code>M_cancel</code> canónico	:51-58
§8.3 ★	S_cancel = ECDSA_Sign(priv_ec_doctor, M_cancel)	:69
§8.5	INSERT <code>Cancelacion(manifiesto, S_cancel)</code>	:77-82

§9 SUSTITUCIÓN — emitir nueva versión de una receta

router `routers/recetas_nueva_version.py`

Equivale a §8 + §4 en una sola transacción: cancela la original con motivo

`"sustituida_por_nueva_version"` y emite una nueva con `parent_id` apuntando a la original, con DEK / IV / firma frescas. Líneas :50-128.

5. Controles criptográficos críticos

Seis puntos donde la cripto **decide** si una operación es legítima. Marcados en código con ★ **CONTROL CRIPTOGRÁFICO** ★.

★	Control	Archivo · línea	Garantiza
1	Firma del médico al emitir	<code>recetas_crear.py:104</code>	Origen de autenticidad. Solo el médico con la priv puede firmar.
2	TAG de AES-GCM al descifrar	<code>aes_gcm.py:71-83</code>	Integridad de C, IV, AAD. Detecta cualquier modificación en BD.
3	Verificar firma del médico al dispensar	<code>recetas_dispensar.py:191</code>	Autenticidad del emisor antes de entregar el medicamento.
4	Firma del sello del farmacéutico	<code>recetas_dispensar.py:211</code>	Sello digital de entrega — equivalente al sello en papel.
5	Pertenencia priv ↔ pub registrada	<code>bundle.py:54-71</code>	Multi-factor implícito: password sola no autoriza firmar nada.
6	Firma de cancelación	<code>recetas_cancelar.py:69</code>	No-repudio. Imposible cancelar suplantando al médico vía SQL.

Demo en vivo: `grep -rn "★ CONTROL" --include="*.py" .` lista los seis con archivo y línea para abrirlos delante del evaluador.

6. Propiedades garantizadas

Propiedad	Cómo se garantiza	Comportamiento si falla
Confidencialidad de R	AES-128-GCM bajo DEK efímera + RSA-OAEP solo abre el destinatario.	Unwrap o TAG fallan → 400.
Integridad de C, IV, AAD	TAG de 128 bits de GCM.	<code>InvalidTag</code> → 400.
Autenticidad del médico	ECDSA-Verify sobre R con pub registrada.	400 antes de dispensar.
Autenticidad del farmacéutico	ECDSA-Sign sobre Sello + verify público.	Endpoint <code>/verificar</code> lo expone.
No-repudio de cancelación	<code>S_cancel</code> persistido junto a <code>M_cancel</code> .	Auditable post-mortem.
AAD vincula metadatos públicos	id_receta, id_doctor, id_paciente, fecha, dispensaciones_permitidas en AAD.	Imposible "mover" un criptograma de fila.
DEK efímera	<code>new_dek()</code> por receta, scrub a ceros tras envolver.	El servidor no puede releer.
Multi-factor implícito	Pertenencia priv → pub validada en cada operación.	Password robada sin priv = 403.
Errores genéricos	Único mensaje <i>"INTEGRIDAD comprometida o firma inválida"</i> .	No hay oráculo de padding.
Audit transaccional	<code>audit.registrar</code> antes del commit.	Imposible operar sin huella.

7. Material privado del sistema

Material	Dueño	Dónde vive	Cómo se generó
Priv EC del usuario (paciente / médico / farm / admin)	el usuario	fuera del servidor — descarga única en registro	<code>keys.py:30</code> <code>generar_par_ecdsa</code>
Priv RSA del usuario	el usuario	fuera del servidor	<code>keys.py:38</code> <code>generar_par_rsa</code> (RSA-2048, e=65537)
Priv EC de la CA interna	servidor	<code>ca/ca_key.pem</code> (chmod 600, sin password)	<code>ca.py:37</code> on-first-boot, autofirmada SHA-256, vigencia 10 años
JWT_SECRET (HMAC)	servidor	<code>.env</code> (mín. 32 chars)	configurado manualmente, validado en arranque
Pubs de todos los usuarios	servidor	tabla <code>usuarios</code> (<code>pub_ec_pem</code> , <code>pub_rsa_pem</code>)	persistidas al registrar
Criptogramas + firmas	servidor	tablas <code>recetas</code> , <code>eventos_dispensacion</code> , <code>cancelaciones</code>	producidas por los flujos

El servidor NUNCA almacena: privadas de usuarios, password en claro, ni la DEK más allá de la operación en curso.

7.1 Generación de la CA interna (`services/crypto/ca.py`)

1. Al primer uso (emisión del primer cert), `_ensure_ca()` comprueba si existe `ca/ca_key.pem`.
2. Si no existe, genera `ec.generate_private_key(SECP256R1)`.
3. Construye un X.509 v3 autofirmado con `BasicConstraints(ca=True, pathLength=0)` y `KeyUsage(key_cert_sign=True, crl_sign=True)`.
4. Vigencia: 10 años (`now + 3650 días`).
5. Lo firma a sí mismo con SHA-256.
6. Persiste a disco: priv en `ca/ca_key.pem` con `chmod 0o600` ; cert en `ca/ca_cert.pem`.

8. Cómo correr las pruebas

La suite de tests está en `tests/test_cripto.py` y cubre **21 escenarios** — caminos felices y todos los caminos negativos de las primitivas. Está escrita como script Python ejecutable, sin dependencia de pytest.

8.1 Pre-requisitos

- Virtualenv activo en `eprescriptions-backend/venv` con `cryptography` instalado (ya viene en `requirements.txt`).
- NO requiere base de datos ni servidor levantado — los tests solo tocan primitivas en memoria.

8.2 Ejecución

```
cd eprescriptions-backend
CRYPTO_LOG=0 ./venv/bin/python tests/test_cripto.py
```

La variable `CRYPTO_LOG=0` silencia los banners visuales del logger durante los tests (si se omite, cada operación cripto imprime su banner — útil pedagógicamente, ruidoso para tests).

8.3 Salida esperada

```
— ECDSA P-256 + SHA3-256 —
✓ firma A → verifica con pub A (camino feliz)
✓ firma A → verifica con pub B → DEBE FALLAR (clave cruzada)
✓ mensaje alterado tras firmar → DEBE FALLAR
✓ firma corrupta (un bit flip en base64) → DEBE FALLAR
✓ firma vacía / mal formada → DEBE FALLAR sin lanzar excepción

— AES-128-GCM (clave 128 bits, TAG 128 bits) —
✓ DEK efímera mide 16 bytes (= 128 bits)
✓ IV mide 12 bytes (= 96 bits, tamaño nativo de GCM)
✓ encrypt → decrypt roundtrip (camino feliz)
✓ clave de 32 bytes → DEBE LANZAR ValueError (forzamos AES-128)
✓ TAG manipulado → DEBE FALLAR (InvalidTag)
✓ ciphertext alterado (1 bit) → DEBE FALLAR
✓ AAD alterado → DEBE FALLAR (TAG vincula AAD al cuerpo)
✓ IV cambiado → DEBE FALLAR
✓ DEK distinta → DEBE FALLAR

— RSA-OAEP-SHA256 / MGF1-SHA256 —
✓ wrap → unwrap roundtrip (camino feliz)
✓ unwrap con priv distinta → DEBE FALLAR
✓ ciphertext del wrap alterado → DEBE FALLAR

— Esquema híbrido (firma + cifrado + envoltura) —
✓ emisión + consulta del paciente: TODO verifica
✓ paciente equivocado (otra priv RSA) → wrap no abre
✓ médico suplantado (otra pub EC) → firma no verifica
✓ atacante cambia AAD para 'mover' la receta → TAG falla
```

```
Resultado:  21 OK    0 FAIL    de 21 tests
```

8.4 Qué demuestra cada bloque

- **ECDSA**: solo la pub correcta verifica; mensaje, firma o codificación alterada → falla.
- **AES-128-GCM**: tamaños correctos; cualquier modificación de TAG, ciphertext, AAD, IV o DEK → `InvalidTag`.
- **RSA-OAEP**: solo la priv pareja descifra; wrap alterado → falla.
- **Híbrido**: simula §4 + §5 completos, incluyendo intentos de suplantación de médico, paciente equivocado, y cambio de AAD para "mover" la receta.

Argumento de cierre: 21/21 OK significa que las primitivas funcionan en sus caminos felices y rechazan correctamente todos los modos de fallo conocidos — no basta con que la cripto funcione cuando todo es correcto, hay que demostrar que detecta cuando no lo es.

9. Cómo ver el flujo en vivo

Cada primitiva criptográfica imprime un banner por `stderr` cuando se ejecuta. Esto permite mostrar en demo el orden exacto de operaciones que ocurren al hacer una receta.

9.1 Levantar el backend

```
cd eprescriptions-backend
source venv/bin/activate
uvicorn main:app --reload
```

9.2 Hacer una receta desde el frontend (en otra terminal)

```
cd eprescriptions-frontend
npm run dev
# Navegar a http://localhost:5173, hacer login como médico
# y crear una receta. La consola de uvicorn mostrará los banners.
```

9.3 Banners que verás en consola al crear una receta

► FLUJO CRIPTOGRÁFICO – §4 CREAR RECETA

[§4 CREAR · paso 0] validar pertenencia de la llave EC del médico
[§4 CREAR · paso 1] serializar R canónico
[§4 CREAR · paso 2] FIRMAR R con ECDSA P-256 + SHA3-256

[CRYPTO] FIRMA · ECDSA P-256 / SHA3-256

Operación: generar firma digital
Mensaje: 234 bytes (hash interno SHA3-256)
Salida: MEYC...== (DER → base64)

[§4 CREAR · paso 3] construir AAD (metadatos autenticados)
[§4 CREAR · paso 4] CIFRAR R con AES-128-GCM + ENVOLVER DEK con RSA-OAEP

[CRYPTO] CIFRADO · AES-128-GCM (clave 128 bits, TAG 128 bits)

Plaintext: 234 bytes
AAD: 145 bytes (autenticado, NO cifrado)
IV: 12 bytes (96 bits, único por operación)
DEK: 16 bytes (128 bits, efímera)
TAG: 16 bytes

[CRYPTO] RSA-OAEP · WRAP (encapsular DEK) ← una vez por destinatario

...

Para silenciar el logger (p.ej. en producción): `export CRYPTO_LOG=0`.

10. Preguntas probables y respuestas en código

Pregunta	Respuesta + ubicación
¿Por qué AES-128 y no AES-256?	Suficiente seguridad y aceleración hardware. Validado en <code>aes_gcm.py:39-42</code> ; <code>new_dek()</code> retorna 16 B (<code>:25-29</code>).
¿Por qué SHA3 y no SHA-256 en ECDSA?	<code>signatures.py:32, :43</code> . SHA3 es familia distinta a SHA-2, sin extensión de longitud.
¿Cómo previenen reuso de IV?	<code>aes_gcm.py:21-22</code> : <code>new_iv()</code> con <code>secrets.token_bytes(12)</code> en cada operación. Las recetas son independientes.
¿Y si modifican la receta cifrada en BD?	El TAG falla. Test "ciphertext alterado" en <code>test_cripto.py</code> .
¿Y si cambian <code>medico_id</code> en la columna SQL?	El AAD lo incluye (<code>canonical_receta.py:62</code>), el TAG falla. Test "AAD alterado".
¿Cómo aseguran que la receta la firmó realmente el médico?	<code>recetas_dispensar.py:191</code> <code>ecdsa_verify(medico.pub_ec_pem, R, firma_doctor)</code> . Test "clave cruzada".
¿Por qué OAEP y no PKCS#1 v1.5?	OAEP es CCA-seguro; PKCS#1 v1.5 sufre Bleichenbacher. Padding configurado en <code>rsa_oaep.py:14-18</code> .
¿Cómo evitan que un atacante con password robada acceda?	<code>bundle.py:54-71</code> . Sin la priv que derive a la pub registrada, 403.
¿Qué hace el AAD?	Va en claro pero el TAG lo autentica. Construido en <code>canonical_receta.py:53-69</code> . Impide "mover" un criptograma a otra fila.
¿La DEK persiste?	No. <code>receta_cifrado.py:48-49</code> la sobrescribe con ceros. <code>recetas_dispensar.py</code> hace lo mismo en cada salida.
¿Qué pasa si el TAG falla?	Mensaje genérico <i>"INTEGRIDAD comprometida o firma inválida"</i> (<code>receta_descifrado.py:20</code>). Anti-oráculo.
¿Por qué Argon2id y no bcrypt?	Memory-hard vs CPU-hard. Argon2id ganó el PHC y resiste GPU/ASIC. <code>argon2_pw.py:8</code> .
¿Cómo se firman los certificados?	CA interna ECDSA P-256 autofirmada en <code>ca.py:37</code> , vigencia 10 años. Emite hojas con <code>KeyUsage</code> según uso (firma vs cifrado).
¿Y si me roban el disco del servidor entero?	Las recetas siguen ilegibles: la DEK está envuelta con la pub RSA del paciente, cuya priv nunca tocó ese disco.
¿Cómo prueban que la cripto está bien?	<code>tests/test_cripto.py</code> — 21 tests, todos los caminos negativos. <code>CRYPTO_LOG=0 ./venv/bin/python tests/test_cripto.py</code> .

Pregunta	Respuesta + ubicación
¿Cómo se ve el flujo en vivo?	<code>uvicorn main:app --reload</code> . Cada operación cripto imprime banner. Implementado en <code>crypto_log.py</code> .

11. Cheatsheet de comandos

```
# Listar los 6 controles cripto centrales
grep -rn "★ CONTROL" --include="*.py" eprescriptions-backend/

# Mapa de archivos cripto-relevantes
ls eprescriptions-backend/services/crypto/      # primitivas
ls eprescriptions-backend/services/             # composición de dominio
ls eprescriptions-backend/routers/              # casos de uso

# Correr la suite de tests negativos
cd eprescriptions-backend
CRYPTO_LOG=0 ./venv/bin/python tests/test_cripto.py

# Ver el flujo en vivo (en una terminal)
uvicorn main:app --reload
# (en otra terminal: hacer una receta desde el frontend)
# Los banners aparecen en stderr de uvicorn en orden:
#   FIRMA → CIFRADO → WRAP → WRAP → ...

# Localizar un algoritmo concreto por nombre
grep -rn "ECDSA\|SHA3_256" services/crypto/
grep -rn "AESGCM"          services/crypto/
grep -rn "OAEP"            services/crypto/

# Silenciar el logger en producción
export CRYPTO_LOG=0

# Verificar configuración del JWT secret
grep "JWT_SECRET" .env      # debe existir, ≥ 32 chars
```


Frase de cierre para la defensa:

El sistema garantiza confidencialidad por AES-128-GCM bajo claves efímeras envueltas con RSA-OAEP, integridad por el TAG de GCM ligado al AAD, autenticidad por ECDSA P-256 sobre SHA3-256, y no-repudio porque cada firma queda persistida en BD. La separación entre primitivas (`services/crypto/`), composición de dominio (`services/`) y casos de uso (`routers/`) hace que cualquier algoritmo se pueda cambiar tocando un solo archivo — como demostramos al pasar de AES-256 a AES-128 modificando una sola primitiva, sin tocar el frontend ni la lógica de negocio.